

PREPGO TECHNICAL SYSTEM SPECIFICATION

PrepGo

AI-Powered Interview Preparation & Assessment Platform
Complete Architectural Blueprint, System Workflow & Detailed
Codebase File Catalog

Platform Name:	PrepGo (Placement Readiness Ecosystem)
Architecture:	MERN Stack (MongoDB, Express, React 19, Node.js) + Multi-Language Sandbox Engine
UI Design System:	Modern Green & White Corporate Theme
Supported Compilers:	Python 3.12, C (GCC 11), C++17 (G++ 11), Java 11 (OpenJDK)
Documentation Date:	October 2026
Status:	Production Ready — End-to-End Verified

1. Project Executive Summary & Description

1.1 What is PrepGo?

PrepGo is a specialized, end-to-end interview preparation and placement assessment web application built for college engineering students, internship aspirants, and software engineering candidates. The platform mirrors rigorous multi-stage campus placement hiring processes conducted by top product startups, tech giants, and IT consulting corporations.

PrepGo bridges the gap between passive learning and real-world placement performance by integrating adaptive timed testing, live code evaluation across multiple programming languages, automated testcase validation, performance tracking, diagnostic placement readiness scoring, and milestone achievements.

1.2 The Complete End-to-End Flow

The platform guides candidates through an integrated, professional placement journey:

PUBLIC LANDING PAGE (/)

↓ User initiates preparation via "Get Started" or "Login"

AUTHENTICATION FLOW (/login , /register , /verify-email , /forgot-password)

↓ Dual authentication: Secure Email/Password with 6-digit OTP delivery OR 1-click Google OAuth 2.0

CENTRAL CANDIDATE DASHBOARD (/home)

↓ Real-time personalized candidate greeting with quick-navigation cards

ROUND 1 — APTITUDE & REASONING (/aptitude)

↓ Difficulty selection (Easy, Medium, Hard) → Timed 20-question dynamic screening test

ROUND 1 RESULT & REVIEW (/aptitude/result , /aptitude/review)

↓ Evaluation metrics (Score/60, Accuracy %, Performance Badge) with option to review answers or proceed

ROUND 2 — TECHNICAL INTERVIEW & DSA (/dsa)

↓ Track selection → Multi-language Coding IDE with live testcase execution (Python, C, C++, Java)

ROUND 2 RESULT & PLACEMENT READINESS (/dsa/result)

↓ Algorithmic scoring, target company tier recommendation, topic strengths & improvement roadmaps

CANDIDATE PROFILE & PREFERENCES (/profile)

↓ Manage education, target job role, skill level, preferred coding language, and contact details

PROGRESS DASHBOARD (/progress)

↓ Real MongoDB attempt metrics: total assessments, questions solved, accuracy, round-wise stats

DETAILED PERFORMANCE REPORTS (/reports)

↓ Difficulty-wise performance breakdown (Easy, Medium, Hard) and chronological attempt history log

MILESTONES & ACHIEVEMENTS (/achievements)

↓ Activity-based badges earned strictly through verified test completions and coding submissions

ROUND 3 — HR INTERVIEW (Roadmap / Coming Soon)

↓ Preserved as non-disruptive roadmap milestone for conversational behavioral preparation

1.3 Core Technology Stack

LAYER	TECHNOLOGIES / LIBRARIES	PURPOSE
Frontend Client	React 19, Vite 8, React Router v7, Axios	Single-page application, client-side routing, high-performance compilation, responsive layouts.
Backend Server	Node.js (v24), Express 5, CORS, Dotenv	RESTful API microservices, route dispatching, request validation, authentication pipelines.
Database & Storage	MongoDB Atlas, Mongoose 9 ODM	Cloud document database, schema validation, indexing, relational cross-referencing.
Authentication & Security	BcryptJS (10 salt rounds), JSON Web Tokens (JWT), Google Auth Library	Encrypted credential storage, stateless session tokens, Google OAuth 2.0 ID token verification.
Communications	Nodemailer (Gmail SMTP integration)	Automated HTML email delivery for 6-digit registration OTPs and password reset codes.
Sandbox Code Execution	Node.js <code>child_process</code> , GCC 11, G++ 11, OpenJDK 11, Python 3.12	Multi-language compilation and execution sandbox with stdin/stdout piping and timeout defense.

2. Detailed Codebase File Catalog (Every Corner)

The PrepGo repository is cleanly decoupled into an Express/Node backend architecture and a modern React/Vite frontend application. Below is the comprehensive audit and technical description of every file in the project.

2.1 Root Level Files

FILE PATH	ROLE	TECHNICAL CONTENTS & RESPONSIBILITIES
<code>.gitignore</code>	Repository Hygiene	Prevents sensitive environment files (<code>.env</code> , <code>.env.local</code>), build artifacts (<code>dist/</code>), process logs (<code>*.log</code>), and <code>node_modules/</code> from being checked into version control.
<code>README.md</code>	Documentation	Project documentation detailing prerequisites, installation commands, architecture overview, and operational procedures.

2.2 Backend Architecture & Files (`/backend`)

FILE PATH	MODULE	TECHNICAL BREAKDOWN & FUNCTIONALITY
<code>backend/server.js</code>	Server Entry Point	The primary HTTP application bootstrap. Configures environment variables via <code>dotenv</code> , initializes database connection via <code>connectDB()</code> , attaches middleware (<code>cors</code> , <code>express.json</code>), mounts dedicated API route routers (<code>/api/auth</code> , <code>/api/aptitude</code> , <code>/api/dsa</code>), exposes health check endpoint <code>GET /</code> , and binds the server to port <code>5000</code> .
<code>backend/package.json</code>	Backend Config	Defines CommonJS module configuration, start scripts (<code>npm start</code> , <code>npm run dev</code>), and pinned production dependencies: <code>bcryptjs</code> , <code>cors</code> , <code>dotenv</code> , <code>express</code> , <code>google-auth-library</code> , <code>jsonwebtoken</code> , <code>mongoose</code> , and <code>nodemailer</code> .
<code>backend/config/db.js</code>	Database Connection	Configures asynchronous connection to MongoDB Atlas using Mongoose. Implements connection logging with active host recognition, error interception, and fatal exit handling upon database failure.
<code>backend/models/User.js</code>	Mongoose Model	Schema definition for candidate identity. Contains core authentication fields (<code>name</code> , <code>email</code> , <code>password</code> , <code>googleId</code> , <code>isVerified</code>), email verification OTP state (<code>otp</code> , <code>otpExpiry</code> , <code>lastOtpSentAt</code>),

FILE PATH	MODULE	TECHNICAL BREAKDOWN & FUNCTIONALITY
		password reset OTP state (<code>resetOtp</code> , <code>resetOtpExpiry</code> , <code>lastResetOtpSentAt</code>), candidate profile attributes (<code>phone</code> , <code>education</code> , <code>targetRole</code> , <code>skillLevel</code> , <code>preferredLanguage</code>), and automatic timestamps.
<code>backend/models/AptitudeQuestion.js</code>	Mongoose Model	Schema for Round 1 assessment questions. Defines <code>question</code> text, exactly 4 validated <code>options</code> (via custom validator), <code>correctAnswer</code> string, <code>difficulty</code> tier (<code>easy</code> , <code>medium</code> , <code>hard</code>), <code>category</code> (<code>quantitative</code> , <code>logical</code> , <code>verbal</code>), and default question marks (3 marks).
<code>backend/models/AptitudeAttempt.js</code>	Mongoose Model	Stores completed aptitude test evaluation records. Contains reference <code>userId</code> (linked to <code>User</code>), selected <code>difficulty</code> , submitted <code>answers</code> map, calculated <code>score</code> (out of 60), computed <code>percentage</code> , and completion timestamp.
<code>backend/models/DSAQuestion.js</code>	Mongoose Model	Schema for technical programming challenges. Contains <code>title</code> , <code>description</code> , <code>difficulty</code> , <code>topic</code> , structured <code>examples</code> (input, output, explanation), <code>constraints</code> array, language-specific <code>starterCode</code> templates (Python, C, C++, Java), reference solutions, and <code>testCases</code> with hidden flag defense (<code>isHidden</code>).
<code>backend/models/DSAAttempt.js</code>	Mongoose Model	Stores evaluated Round 2 coding assessment attempts. Records <code>userId</code> , <code>difficulty</code> , <code>language</code> used, submitted <code>code</code> map, per-problem test results <code>answers</code> , total <code>score</code> (out of 60), <code>percentage</code> , <code>passedQuestions</code> count, <code>failedQuestions</code> count, <code>timeUsed</code> , and generated <code>readinessAnalysis</code> .
<code>backend/controllers/authController.js</code>	Auth & Analytics Logic	Comprehensive controller encompassing: <code>registerUser</code> : Validates inputs, hashes password with bcrypt, generates 6-digit OTP, creates unverified user, and dispatches verification email. <code>verifyEmail</code> : Validates OTP against database, enforces 10-minute expiry, marks account verified.

FILE PATH	MODULE	TECHNICAL BREAKDOWN & FUNCTIONALITY
		<code>resendOTP</code> : Issues fresh OTP with 60-second cooldown rate-limiting. <code>loginUser</code> : Authenticates credentials, verifies active verification status, issues 7-day JWT token, and returns user profile payload. <code>forgotPassword</code> & <code>verifyResetOTP</code> & <code>resetPassword</code> : Secure password recovery pipeline. <code>getUserProfile</code> & <code>updateUserProfile</code> : Candidate profile attributes management with token verification. <code>getUserAnalytics</code> : Aggregates real MongoDB attempts from <code>AptitudeAttempt</code> and <code>DSAAttempt</code>, computes real accuracy, difficulty tier distributions, and awards activity-based achievements.
<code>backend/controllers/googleAuthController.js</code>	OAuth Controller	Handles Google Sign-In via Google Auth Library. Exposes <code>getGoogleClientId</code> to provide public client IDs to the frontend, verifies client Google ID tokens with Google's public keys, retrieves user payload (email, name, picture), creates or updates user in MongoDB, and issues signed PrepGo JWT session tokens.
<code>backend/controllers/aptitudeController.js</code>	Aptitude Logic	Controls Round 1 screening. Contains <code>getQuestionsByDifficulty</code> (samples 20 questions excluding answers to prevent cheating), <code>submitAptitudeTest</code> (grades answers against database, computes marks and percentage, saves attempt), and <code>getLatestAttempt</code> (retrieves candidate's most recent aptitude result).
<code>backend/controllers/dsaController.js</code>	DSA & Code Sandbox	Core technical execution module: <code>executeCode</code> : Spawns sandboxed processes in temporary directories. Compiles C (GCC), C++17 (G++), and Java 11 (Javac) with dedicated 12s/15s compilation timeouts, and executes with 4s runtime limits against standard input. <code>areOutputsEquivalent</code> : Normalizes and compares actual vs expected outputs,

FILE PATH	MODULE	TECHNICAL BREAKDOWN & FUNCTIONALITY
		stripping formatting nuances and handling floating-point tolerances. <code>runDSACode</code> : Runs candidate code against visible sample testcases for live feedback. <code>submitDSATest</code> : Grades code against public and hidden test cases, computes scores, generates Placement Readiness Analysis, and saves attempts.
<code>backend/routes/authRoutes.js</code>	Route Definitions	Maps HTTP methods to controller functions for authentication (<code>/register</code> , <code>/login</code> , <code>/verify-email</code> , <code>/resend-otp</code>), Google OAuth (<code>/google</code> , <code>/google-client-id</code>), password recovery (<code>/forgot-password</code> , <code>/reset-password</code>), profile updates (<code>/profile</code>), and unified analytics (<code>/progress</code> , <code>/reports</code> , <code>/achievements</code>).
<code>backend/routes/aptitudeRoutes.js</code>	Route Definitions	Declares API endpoints for aptitude test operations: <code>GET /:difficulty</code> (question sampling), <code>POST /submit</code> (test evaluation), and <code>GET /latest-attempt</code> .
<code>backend/routes/dsaRoutes.js</code>	Route Definitions	Declares endpoints for technical DSA operations: <code>GET /:difficulty</code> (random problem selection), <code>POST /run</code> (sample execution), <code>POST /submit</code> (assessment grading), and <code>GET /latest-attempt</code> .
<code>backend/utils/sendEmail.js</code>	Email Transport Utility	Configures Nodemailer transporter using Gmail SMTP service. Renders styled HTML email templates with PrepGo branding for 6-digit email verification OTPs and password reset verification codes.
<code>backend/seedAptitude.js</code>	Database Seed Script	Automated database population utility that populates MongoDB with 60 curated quantitative, logical reasoning, and verbal aptitude questions categorized across Easy, Medium, and Hard difficulty tiers.

FILE PATH	MODULE	TECHNICAL BREAKDOWN & FUNCTIONALITY
<code>backend/seedDSA.js</code>	Database Seed Script	Populates MongoDB with 18 curated Data Structures and Algorithms problems (Easy, Medium, Hard). Configured with clean starter templates only (no preloaded solutions) and comprehensive public and hidden test cases.
<code>backend/testDatabase.js</code>	Diagnostic Utility	Standalone script to connect to MongoDB Atlas, verify model compilation, and output document counts across all collections.

2.3 Frontend Architecture & Files (/frontend)

FILE PATH	TYPE	TECHNICAL BREAKDOWN & USER EXPERIENCE
frontend/index.html	HTML Template	Host HTML document. Configures Google fonts (Inter , Poppins), loads Google Identity Services client script for 1-click Google Sign-In, and mounts the React root DOM node.
frontend/package.json	Dependencies	Configures modern client dependencies: React 19, ReactDOM 19, React Router v7, Axios, and Vite build tooling.
frontend/vite.config.js	Build Config	Vite configuration enabling React plugin, fast hot-module replacement (HMR), and optimized production asset chunking.
frontend/src/main.jsx	React Bootstrap	Initializes React application root, wraps application inside React Router's BrowserRouter , and imports global styling (theme.css).
frontend/src/App.jsx	Central Router	Declarative routing hub declaring 17 application routes covering the complete placement preparation pipeline from landing, authentication, dashboard, rounds 1 & 2, results, profile, progress, reports, and achievements.
frontend/src/theme.css	Design System	Central design tokens defining CSS custom properties: --primary-green (#16a34a), --primary-green-dark (#15803d), --primary-green-subtle (#f0fdf4), surfaces, borders, card shadows, and typography tokens.
frontend/src/LandingPage.jsx	Public Component	Clean, minimalist public hero landing page. Features PrepGo branding, headline "Prepare Smarter. Interview Better.", descriptive placement subtitle, and call-to-action buttons ("Get Started →", "Login").
frontend/src/LandingPage.css	Styling	Modern styling for landing page with soft green gradients, centered typography, responsive container queries, and mobile adaptations.

FILE PATH	TYPE	TECHNICAL BREAKDOWN & USER EXPERIENCE
<code>frontend/src/Login.jsx</code>	Auth Component	Split-screen authentication portal. Left column presents the brand badge and the natural 16:9 illustration (<code>/prepgo-login-image.jpeg</code>). Right column presents the login card with email/password authentication, Google OAuth login, Remember Me checkbox, and links to registration and password reset.
<code>frontend/src/Login.css</code>	Styling	Styling for login portal. Manages proportional scaling of hero illustration with <code>object-fit: contain</code> , ambient green glow bubbles, form input focus states, and mobile collapsing.
<code>frontend/src/Register.jsx</code>	Auth Component	Candidate sign-up interface. Captures full name, email, and password. Performs validation, invokes <code>POST /api/auth/register</code> , and redirects candidates to email OTP verification upon code dispatch.
<code>frontend/src/Register.css</code>	Styling	Registration card styling consistent with the Green + White design language, featuring interactive input icons and error feedback banners.
<code>frontend/src/VerifyEmail.jsx</code>	Auth Component	6-digit OTP email verification component. Includes 60-second resend cooldown timer, dynamic digit paste/input management, automatic error alerts, and seamless redirection to login upon verification.
<code>frontend/src/VerifyEmail.css</code>	Styling	Digit input box styling, pulse animations, cooldown timer indicators, and success confirmation views.
<code>frontend/src/ForgotPassword.jsx</code>	Auth Component	Password recovery initiation screen. Accepts candidate email, invokes <code>POST /api/auth/forgot-password</code> , and navigates to reset OTP verification.
<code>frontend/src/VerifyResetOTP.jsx</code>	Auth Component	Validates 6-digit password reset OTP against backend, confirming identity before allowing password replacement.
<code>frontend/src/ResetPassword.jsx</code>	Auth Component	New password creation interface with password confirmation matching, strength checks, and direct redirection to login upon successful reset.
<code>frontend/src/Home.jsx</code>	Core Dashboard	Candidate home portal: Top navigation header with brand badge, module shortcuts (01 Round 1, 02 Round 2, 03 Round 3, Profile, Progress, Reports, Achievements), user avatar, and logout.

FILE PATH	TYPE	TECHNICAL BREAKDOWN & USER EXPERIENCE
		- Personalized greeting ("Good Morning, [Name] 🙌") derived dynamically from authenticated user state. - Interactive 5-module horizontal card row: Round 1 (Aptitude), Round 2 (Technical DSA), Round 3 (HR - Coming Soon), Profile & Settings, Progress Dashboard. - Responsive mobile navigation drawer.
<code>frontend/src/Home.css</code>	Styling	Dashboard grid and card layout styling, hover elevations, status pills, and desktop single-row module presentation.
<code>frontend/src/AptitudeHome.jsx</code>	Round 1 Entry	Aptitude track selector presenting Easy (15m, 60 marks), Medium (20m, 60 marks), and Hard (25m, 60 marks) screening levels.
<code>frontend/src/AptitudeInstructions.jsx</code>	Round 1 Rules	Assessment briefing screen outlining question distribution, non-negative marking guidelines, and timer rules prior to test commencement.
<code>frontend/src/AptitudeTest.jsx</code>	Round 1 Workspace	Complete assessment workspace: fetches 20 dynamic questions from MongoDB, runs countdown timer with auto-submit on expiry, manages question navigation, maintains question palette status (answered vs unanswered), and handles submit confirmation modals.
<code>frontend/src/AptitudeTest.css</code>	Styling	Two-column workspace layout with question card on the left, 20-item question palette on the right, and modal overlays.
<code>frontend/src/AptitudeResult.jsx</code>	Round 1 Result	Evaluation presentation featuring circular percentage score ring, performance badge, score summary (Score, Correct, Wrong), and action buttons: "Proceed to Round 2 (Technical DSA) →", "Review Answers", "View Progress", and "Dashboard".
<code>frontend/src/ReviewAnswers.jsx</code>	Round 1 Review	Question-by-question post-assessment review page highlighting correct answers in green and incorrect candidate choices in red. Protected with null state guards.
<code>frontend/src/DSAHome.jsx</code>	Round 2 Entry	Technical interview track selection portal detailing Easy, Medium, and Hard coding modes, expected time, marks, and relevant algorithmic topic tags.

FILE PATH	TYPE	TECHNICAL BREAKDOWN & USER EXPERIENCE
<code>frontend/src/DSAInstructions.jsx</code>	Round 2 Rules	Technical coding guidelines explaining multi-language support (Python, C, C++, Java), standard input/output formatting, and compiler specifications.
<code>frontend/src/DSATest.jsx</code>	Round 2 Workspace	Full-featured coding IDE: • Problem statement pane with constraints, formatted examples, and hints modal. • Language selector (Python, C, C++, Java) preloading clean starter templates only (no complete answers). • Code editor with line numbers and syntax formatting. • Testcase execution console running code against sample test cases via <code>POST /api/dsa/run</code>. • Assessment submission via <code>POST /api/dsa/submit</code> against hidden test cases with execution time metrics.
<code>frontend/src/DSATest.css</code>	Styling	Professional dark/light split IDE workspace styling, code editor container, terminal output tabs, and timer controls.
<code>frontend/src/DSAResult.jsx</code>	Round 2 Result	Technical evaluation report featuring circular score visual, problem breakdown cards, and PrepGo's signature Placement Readiness Analysis : diagnostic mapping algorithmic strengths, improvement areas, recommended company hiring tier (e.g. Tier 1 Product vs High-Growth Startups), and suggested next topics.
<code>frontend/src/Profile.jsx</code>	Candidate Profile	Candidate profile management interface allowing editing and saving of personal details (phone, education, target role, skill level, preferred language) with persistent database synchronization. Includes navigation tabs to Progress, Reports, and Achievements.
<code>frontend/src/Profile.css</code>	Styling	Two-column profile layout styling with avatar identity card on the left and form inputs on the right.

FILE PATH	TYPE	TECHNICAL BREAKDOWN & USER EXPERIENCE
<code>frontend/src/Progress.jsx</code>	Analytics Component	Progress dashboard consuming real MongoDB evaluation data. Displays 4 key KPI cards (Total Assessments, Questions Solved, Round 1 Best Score, Round 2 Best Score), module performance breakdowns with difficulty pills, quick-link navigation cards, and recent assessment session table.
<code>frontend/src/Progress.css</code>	Styling	Modern Green + White analytics dashboard styling with responsive KPI card grids and table viewports.
<code>frontend/src/Reports.jsx</code>	Reports Component	Comprehensive evaluation reporting component. Presents round-wise analysis cards, difficulty tier distribution cards (Easy, Medium, Hard), and a filterable chronological assessment session history log.
<code>frontend/src/Reports.css</code>	Styling	Styling for report summary cards, tier cards, filter pill buttons, and responsive tabular history views.
<code>frontend/src/Achievements.jsx</code>	Milestones Component	Milestones and badges showcase evaluated strictly against verified database attempts. Displays badge cards with unlock statuses, completion percentages, unlock timestamps, and preparation calls to action.
<code>frontend/src/Achievements.css</code>	Styling	Badge card grid styling, unlocked vs locked card states, progress circle banner, and filter toggles.
<code>frontend/public/prepgo-login-image.jpeg</code>	Static Asset	Reference high-resolution graphic showcasing laptop with PrepGo dashboard, study notes, books, mug, and green ambient accents displayed on the login page.

3. Database Architecture & Data Models

PrepGo utilizes MongoDB Atlas with Mongoose schemas. All data models are strictly scoped to the active modules of the application.

COLLECTION NAME	MODEL CLASS	KEY DOCUMENT FIELDS	RELATIONSHIPS & INTEGRITY
users	User	name, email, password, googleId, phone, education, targetRole, skillLevel, preferredLanguage, isVerified, otp, otpExpiry, resetOtp, resetOtpExpiry	Primary candidate entity. Referenced by AptitudeAttempt.userId and DSAAttempt.userId. Unique index enforced on email.
aptitudequestions	AptitudeQuestion	question, options (array of 4 strings), correctAnswer, difficulty, category, marks	Contains 60 curated questions across Quantitative, Logical, and Verbal topics. Indexed by difficulty.
aptitudeattempts	AptitudeAttempt	userId, difficulty, answers (map), score, percentage, completedAt	Child record of candidate aptitude session. Populated by POST /api/aptitude/submit. Used for analytics and reporting.
dsaQuestions	DSAQuestion	title, description, difficulty, topic, examples, constraints, starterCode (Python, C, C++, Java), testCases (input, expectedOutput, isHidden), marks	Contains 18 curated technical problems across Easy, Medium, and Hard. Indexed by difficulty. Hidden testcases are excluded from client queries.
dsaAttempts	DSAAttempt	userId, difficulty, language, selectedQuestions, code, answers, score, percentage, passedQuestions, failedQuestions, timeUsed, readinessAnalysis, completedAt	Records full technical coding assessment results. Populated by POST /api/dsa/submit. Powers the Progress, Reports, and Achievements modules.

4. Security, Diagnostics & Verification Summary

4.1 Security & Data Protection Standards

- **Password Encryption:** User passwords are encrypted using Bcrypt with 10 salt rounds before storage in MongoDB.
- **Stateless Authentication:** Signed JWT tokens with 7-day expiration authorize protected API endpoints via Bearer headers.
- **Data Isolation & Redaction:** Sensitive attributes (`password` , `otp` , `otpExpiry` , `resetOtp`) are strictly excluded from profile and analytics responses.
- **Sandbox Containment:** Candidate code execution is executed in temporary directories generated on demand via `fs.mkdtempSync` , terminated with strict process execution timeouts, and sanitized upon completion.
- **Starter Code Integrity:** Starter templates provide method skeletons only; complete solutions are never preloaded or exposed.

4.2 Verification Confirmation

VERIFICATION ITEM	STANDARD / TARGET	OBSERVED RESULT	STATUS
Frontend Production Build	<code>npm run build</code>	Vite 8 build completed in 2.23s with 0 errors. Bundled output generated in <code>dist/</code> .	PASSED
Python Code Execution	Python 3.12 sandbox	Evaluated sample and hidden test cases with correct output parsing.	PASSED
C Code Execution	GCC 11 sandbox	Compiled and executed with standard input/output.	PASSED
C++ Code Execution	G++ 11 (C++17) sandbox	Compiled without premature timeout and executed with 100% testcase passing.	PASSED
Java Code Execution	OpenJDK 11 sandbox	Javac compiled and Java runtime executed with 100% testcase passing.	PASSED
Progress & Analytics API	<code>GET /api/auth/progress</code>	Computes exact counts from MongoDB; returns "N/A" for unattempted metrics (0 fake data).	PASSED
Achievements Evaluation	Activity-based badges	Badges awarded strictly from verified attempt counts and score benchmarks.	PASSED

Operational Startup Instructions:

- Backend: `cd backend && npm start` (Runs on port 5000)
- Frontend: `cd frontend && npm run dev` (Accessible on port 5173)

- Access PDF: This document is served publicly at

http://localhost:5173/PrepGo_Project_Documentation.pdf